

## ***Instrucciones en Python***

### **Estructuras de Control: Secuenciales**

#### **Entrada, Salida y Asignación**

##### **a) Entrada: Leer(variable)**

**input:** devuelve el valor ingresado por teclado tal como se lo digita a una variable.

Ejemplos:

```
>>> cod = int (input ("Ingrese el código de producto: "))
>>> print(cod)
270
>>> descrip = input ("ingrese descripción del producto")
>>> print(descrip)
Destornillador eléctrico
```

##### **b) Salida: Mostrar(variable)**

**print:** permite mostrar texto, y valores de variables o expresiones en pantalla.

Ejemplo:

- El texto se delimita entre comillas "", y print admite varios argumentos separados por coma o sin ella:

```
>>> print("Encontrado")
Encontrado
>>> print ("Encontrado", "el producto")
Encontrado el producto
>>> print ("Encontrado" "el producto")
Encontradoelproducto
```

- Insertar salto de línea automático

```
>>> print ("Encontrado")
>>> print ("el producto")
Encontrado
el producto
```

- Incluir variables y/o expresiones

```
>>> print ("Encontrado el codigo ", cod, "del producto ", nombre)
Encontrado el código 230 del producto Destornillador Eléctrico
```

- Uso de cadenas f  

```
>>> print ("Encontrado el código (cod) del producto (nombre)")
Encontrado el codigo 230 del producto Destornillador Eléctrico
```
- Algunos delimitadores: end=" " y sep=" "  

```
>>> print ("Encontrado", end=" ")
>>> print (" el producto")
Encontrado el producto
>>> print ("rojo", "blanco", "azul", sep=",")
rojo, blanco, azul
>>> print ("rojo","blanco"," azul", sep=" - ")
rojo - blanco - azul
```
- Anteponiendo f o F antes de la cadena de caracteres se puede dar formato a las variables con uso de llaves.  

```
>>> anio=2018; tema=" facturación"
>>> f"Resultados del año {anio}{tema}"
```

### c) Sentencia de asignación

**"El contenido de los identificadores definen su tipo"**

#### Asignación simple:

Se utiliza el **operador =**

Ejemplo:

|                               |                                             |
|-------------------------------|---------------------------------------------|
| <code>valor1 = 1</code>       | <code>int ()</code> entero                  |
| <code>valor2 = 2.4</code>     | <code>float()</code> real                   |
| <code>texto = "prueba"</code> | <code>string ()</code> cadena de caracteres |
| <code>constante = 12</code>   | <code>int()</code> entero                   |
| <code>IVA = 21</code>         | constante entera                            |

#### Asignación múltiple:

Se permite poder asignar en una sola instrucción, valores a múltiples variables, por su orden:

```
a, b, c = 'string', 15, True
```

## Estructuras de Control: Decisión Simple

Todas las sentencias compuestas constan del encabezado que comienza con una palabra “reservada” de identificación única y finaliza con : (dos puntos).

El conjunto de sentencias controladas por las mismas, deben ser 1 o más sentencias simples que se declaran separadas por ; (punto y coma) en la misma línea del encabezado o una o más sentencias simples o compuestas indentadas a 4 espacios en las líneas siguientes.

### a) if...

**Python:**

```
if expresión_lógica: sentencia_simple1; sentencia_simple2  
    sentencia3
```

**Sintáxis:**



### b) if ...else...

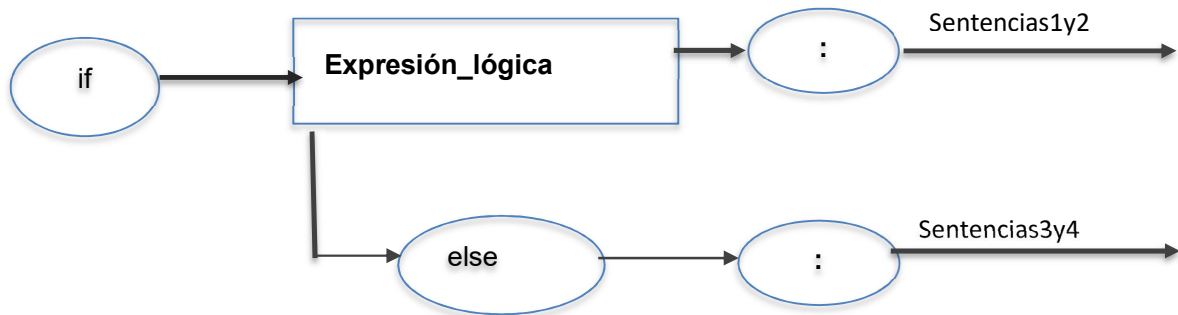
**Python:**

```
if expresión_lógica:  
    sentencia_1  
    sentencia_2  
    ...  
else:  
    sentencia_3  
    sentencia_4  
    ...
```

El conjunto de sentencias 1 y 2 se realizarán cuando expresión\_lógica = true, si es False continúa con la secuencia del programa (sentencia3)

Las mismas deben ser sentencias simples, ya que están separadas por ; y se ejecutarán todas las declaradas

**Sintaxis:**



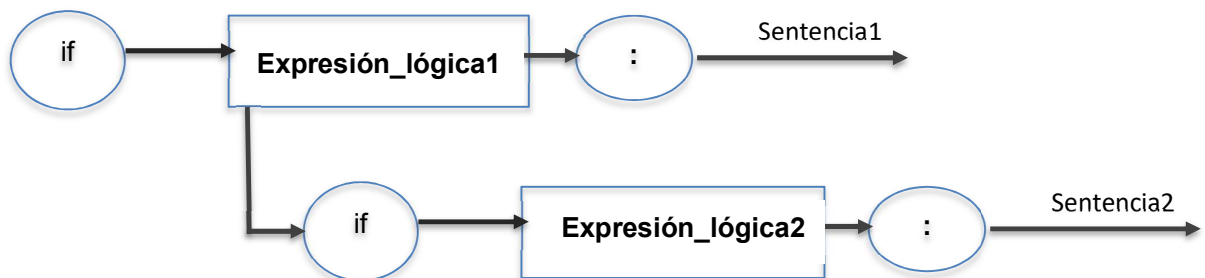
**c) if...if... (If Anidados)**

**Python:**

```
if expresión_lógica1:  
    sentencia_simple1  
    if expresión_lógica2: sentencia_simple2  
sentencia3
```

El conjunto de sentencias se ejecutará cuando la expresión\_lógica1 sea igual a True, si es False continúa la ejecución en el orden lógico del programa (sentencia3). Permite ejecutar sentencias compuestas o simples, pues están indentadas en las líneas posteriores.

**Sintaxis:**



#### d) **if...elif...else...**

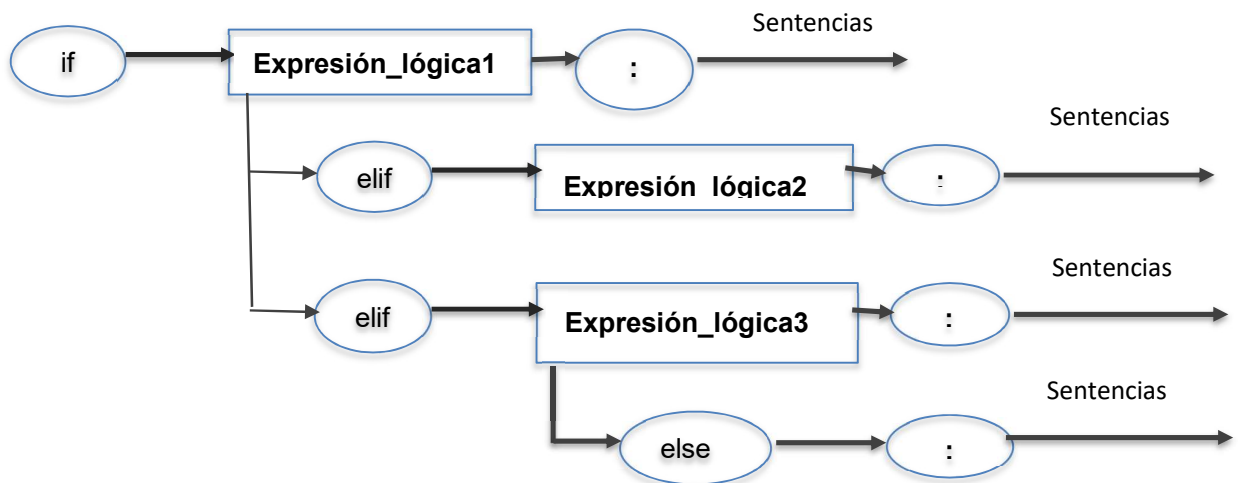
**Python:**

```
if expresion_logica1:  
    sentencia1  
    ...  
    elif expresion_logica2:  
        sentencia2  
        ...  
    elif expresion_logica3:  
        sentencia3  
    ...  
else:  
    sentencia4  
    .  
    .
```

**Sentencia**

**elif** es la abreviatura de **else if**, se aplica cuando tenemos que encadenar varias condiciones lógicas.  
Luego de los : (dos puntos) se ejecutará el bloque indentado si la expresión\_lógica es True.  
Luego evaluará las condiciones encadenadas sucesivas (pueden utilizarse los **elif** que sean necesarios, (hasta cubrir todas las posibilidades de una condición).  
El uso del **else:** es opcional, si se quiere ejecutar un bloque de sentencias si la expresión\_lógica es False.

**Sintáxis:**



Python no posee la **estructura de decisión múltiple**, esta forma se asemeja, pero la expresión\_lógica puede tomar cualquier forma de relación.

## Recursos de consulta

**Sitio Oficial:**

<https://www.python.org/>

**Tutorial:**

<https://docs.python.org/3/tutorial/>

**Referencias del Lenguaje:**

<https://docs.python.org/3/reference/index.html>

**Gramática completa de Python versión 3.6**

<https://docs.python.org/3.6/reference/grammar.html>

<http://www.python.org.ar/>

<http://www.learnpython.org/es/>

<https://www.codecademy.com/es/tracks/python-traduccion-al-espanol-america-latina-clone-1>

**Python para Todos.pdf (licence common) de González Luque, Raúl:**

<http://mundogeek.net/tutorial-python/>